Speech in NLP: Speech-to-Text (STT / ASR)

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 27, 2026

Table of Contents

Introducing the speech recognition task

- **Broad goal.** Given audio, produce exact transcript of words from speaker(s).
- **Verbatim (Exact) Transcription:** Capturing every "um," word fragment, repetition, and disfluency to preserve the true speech signal.
- **Clean (Readable) Transcripts:** Text that omits or normalizes minor speech errors and filler words, yielding more fluent, reader-friendly output.
- **Real-Time, Low-Latency Operation:** For interactive settings (voice assistants, live captions), ASR must process audio as it arrives. Keep end-to-end latency under 200 ms for natural dialogue flow.
- **On-Device and Edge Constraints:** Need to run ASR on devices with no cloud support. Compute and memory may be limited.

Switchboard corpus conversation example

Arc of Recent History

- **In 2010:**
  - ASR, TTS, dialog all used specialized, hard-to-build modeling approaches
  - Industry application of SLU systems limited. ASR "didn't quite work well enough"
- **Today:**
  - ASR, TTS, dialog all use deep learning approaches. Less specialized and better performance
  - Spoken language systems are everywhere!
  - New tools enable building full systems

Automaton:

- Markov 1911
- Turing 1936
- McCulloch-Pitts neuron (1943)
  - mar.bsee.swin.edu.au/cht/net204/lecture10/ann/node1.html
  - www.epfl.ch/mtap/tutorial/english/mcpitts.html
- Shannon (1948) link between automata and Markov models

Human speech processing

- Fletcher at Bell Labs (1920's)

Probabilistic/Information-theoretic models

- Shannon (1948)

Early Recognition: 1920's Radio Rex

../assets/New-Lecture-STT/cs224s_lec9_slide_46.png

Radio Rex toy and box (c. 1920s)

1920's Radio Rex

- Celluloid dog with iron base held within house by electromagnet against force of spring
- Current to magnet flowed through bridge which was sensitive to energy at 500 Hz
- 500 Hz energy caused bridge to vibrate, interrupting current, making dog spring forward
- The sound "e" (ARPAbet [eh]) in Rex has 500 Hz component

ASR: 1950's – Early Speech Recognizers

- **1952: Bell Labs single-speaker digit recognizer**
  - Measured energy from two bands (formants)
  - Built with analog electrical components
  - 2% error rate for single speaker, isolated digits
- **1958:** Dudley built classifier that used continuous spectrum rather than just formants
- **1959:** Denes ASR combining grammar and acoustic probability

- **Hidden Markov Model 1972**
  - Independent application of Baker (CMU) and Jelinek/Bahl/Mercer lab (IBM) following work of Baum and colleagues at IDA
- **ARPA project 1971–1976**
  - 5-year speech understanding project: 1000 word vocab, continuous speech, multi-speaker
  - SDC, CMU, BBN
  - Only 1 CMU system achieved goal
- **1980's +**
  - Annual ARPA "Bakeoffs"
  - Large corpus collection:
    - TIMIT
    - Resource Management
    - Wall Street Journal

Recent ASR Improvements

assets/New-Lecture-STT/cs224s_lec9_slide_52.png

Classical ASR: The Noisy Channel Model

- **Search through space of all possible sentences.**
- **Pick the one that is most probable given the waveform.**

assets/New-Lecture-STT/cs224s_lec9_slide_05.png

The Noisy Channel Model

- What is the most likely sentence out of all sentences in the language L that generated some given **acoustic input** O?
- Treat acoustic input O as sequence of individual observations:

$$O = o_1, o_2, \ldots, o_T$$

- Define a sentence as a sequence of words:

$$W = w_1, w_2, \ldots, w_n$$

The Noisy Channel Model

- **Probabilistic implication:** Pick the highest probability word sequence W:

$$\widehat{W} = \arg\max_{W \in L} P(W \mid O)$$

- **We can use Bayes' rule to rewrite this:**

$$\widehat{W} = \arg\max_{W \in L} \frac{P(O \mid W)P(W)}{P(O)}$$

- Since denominator is the same for each candidate sentence W, we can ignore it for the argmax:

$$\widehat{W} = \arg\max_{W \in L} P(O \mid W)P(W)$$

The Noisy Channel Model

assets/New-Lecture-STT/cs224s_lec9_slide_09.png

Word error rate (WER)

assets/New-Lecture-STT/cs224s_lec9_slide_11.png

Speech Recognition Architecture

assets/New-Lecture-STT/cs224s_lec9_slide_13.png

Classical ASR:HMM-GMM system

assets/New-Lecture-STT/cs224s_lec9_slide_16.png

Hidden Markov Model (HMM) basics

- **Markov assumption:**

$$P(q_t \mid q_1, \ldots, q_{t-1}) = P(q_t \mid q_{t-1})$$

- **Output-independence assumption:**

$$P(o_t \mid O_1^{t-1}, q_1^t) = P(o_t \mid q_t)$$

$Q = q_1, \ldots, q_T$	a set of N states
$A = a_{ij}$:	**transition probability matrix** A, each a_{ij} representing the probability of moving from state i to state j. $\sum_j a_{ij} = 1 \ \forall i$
$O = o_1, \ldots, o_T$	a sequence of T observations, each one drawn from a vocabulary $V = v_1, \ldots, v_W$
$B = b_j(o_t)$	a sequence of **observation likelihoods**, also called *emission probabilities*, each expressing the probability of an observation o_t being generated from a state j
$q_0,\ q_F$	a special **start state** and (**final**) **end state** that are not associated with observations, together with transition probabilities $a_{q_0 i}$ out of the start state and $a_{j q_F}$ into the end state

Hidden Markov Model (HMM) basics

- **Objective function for DNN training: per-frame classification**
- Supervised learning: minimize classification errors at each time frame.
- **Standard choice: Cross entropy loss function**
  - Extension of logistic loss for multiclass problems.
  - For K classes, target label y, input x, model parameters W, b:

$$\mathrm{Loss}(x, y; W, b) = -\sum_{k=1}^{K} \mathbb{I}(y = k) \log f(x)_k$$

$f(x)_k$: predicted probability for class k at input x

- This is a **frame-wise** loss: uses a label for each acoustic frame (from forced alignment).
- Other loss functions are possible, including ones that incorporate full-sequence or word error rate criteria, or deeper integration with HMMs.

- **Feature Extraction:**
  - 39 MFCC features
- **Acoustic Model:**
  - Gaussian mixture models (GMM) to approximate $p(o|q)$
- **Lexicon/Pronunciation Model:**
  - HMM: which phones can follow each other. Sequence constraints from phonetic lexicon
- **Language Model:**
  - N-grams for computing $p(w_i|w_{i-1})$
- **Decoder:**
  - Viterbi algorithm: dynamic programming for combining all pieces to estimate a word sequence from audio

Source: Stanford CS224S, Lecture 9 (2025 Spring), slide 15 (adapted).

Simple HMM: Estimating climate from ice cream observations

You are a climatologist in the year 2799 studying global warming trends

- You can't find any records of the weather in Baltimore, MD for summer of 2008, but you find Jason Eisner's diary! Which lists how many ice-creams Jason ate every date that summer

Our job: Infer the sequence of daily temperatures (HOT vs COLD)

- We do not observe temperature. We observe number of ice creams eaten
- There is a higher probability of eating more ice cream on hot vs cold days
- There is some relationship of next day temperature given current day temperature

Simple HMM: Estimating climate from ice cream observations

assets/New-Lecture-STT/cs224s_lec9_slide_19.png

Audio Feature Extraction: MFCCs

- Human hearing is not equally sensitive to all frequency bands
- Less sensitive at higher frequencies, >1000 Hz
- I.e. human perception of frequency is non-linear

assets/New-Lecture-STT/cs224s_lec9

Mel Filter Bank Processing

assets/New-Lecture-STT/cs224s_lec9_slide_23.png

Gaussians Mixtures for Acoustic Modeling

assets/New-Lecture-STT/cs224s_lec9_slide_27.png

GMMs

assets/New-Lecture-STT/cs224s_lec9_slide_28.png

Training an HMM system (Viterbi)

- **Given our lexicon + HMM structure, and some acoustic model, we can:**
  - Generate the best alignment of HMM states to acoustic observations
- **With an alignment of HMM states to observations:**
  - Build a new acoustic model. Treat current state/obs mapping as training data+labels
  - This acoustic model is hopefully better than previous one
- **Repeat the align → rebuild acoustic model process until convergence**
  - Add parameters / complexity to acoustic model each iteration

Hybrid HMM-DNN ASR

Transition to deep learning in acoustic modeling:
Hybrid HMM-DNN systems

Objective function for DNN training: per-frame classification

- **Supervised learning**, minimize our classification errors
- **Standard choice: Cross entropy loss function**
  - Straightforward extension of logistic loss for binary

$$Loss(x, y; W, b) = -\sum_{k=1}^{K} (y = k) \log f(x)_k$$

- This is a **frame-wise** loss. We use a label for each frame from a forced alignment
- Other loss functions possible. Can get deeper integration with the HMM or word error rate

Neural nets for acoustic modeling goes back to early speech

assets/New-Lecture-STT/cs224s_lec10_slide_10.png

Early neural network approaches

Speech tasks motivated some of the original backpropagation work

assets/New-Lecture-STT/cs224s_lec10_slide_11.

Hybrid systems took over ASR around 2012-2015

- DNN–HMM hybrids (Hinton et al., 2012) rapidly replaced GMM–HMM on large-vocabulary tasks.
- Sets the stage for deeper encoders, sequence training, and later end-to-end models.

assets/New-Lecture-STT/cs224s_lec10_slide_13.png

What's made modern DNNs successful?

- Context-dependent HMM states
- Deeper nets improve on single hidden layer nets
- Hidden unit nonlinearity
- Many more model parameters (scaling up) & large datasets
- Specific depth (e.g. 3 vs 7 hidden layers)
- Fast computers = run many experiments
- Architecture choices (easiest is replacing sigmoid)
- Pre-training does not matter*
  - Initially we thought this was the new trick that made things work

Recurrent DNN hybrid acoustic models

assets/New-Lecture-STT/cs224s_lec10_slide_15.png

Deep recurrent networks

assets/New-Lecture-STT/cs224s_lec10_slide_16.png

How can we improve upon HMM-DNN?

- Lexicon introduces too many pronunciation assumptions. Requires hand engineering.
- Iteratively building HMM systems requires complex "recipes" to progressively improve alignments + acoustic models.
- HMM-DNN systems perform better, but still require the above
  - Can we use deep learning approaches to replace the HMM-based approaches so far?

End-to-end alternative to hybrid HMM–DNN: map audio directly to transcript tokens (characters, phonemes, BPE).

HMM-free recognition with CTC

- DNN approximates $P(a \mid x)$ over output tokens; no separate HMM state topology for alignment.

assets/New-Lecture-STT/cs224s_lec10_slide_25.png

CTC output assumptions

- Each timestep t the model outputs $P(a \mid X_{1:T})$, which can either be an output token (character, phoneme) or "blank".
- *Assume:* Input sequence (length T) $\geq$ output sequence length (safe assumption in ASR).
- **Collapsing rule:** Ignore all blank tokens, and ignore repeated outputs of the same token.

assets/New-Lecture-STT/cs224s_lec10_slide_26.png

CTC collapsing. Sum over all possible alignments

assets/New-Lecture-STT/cs224s_lec10_slide_27.png

Real example. Full utterance with blank token

assets/New-Lecture-STT/cs224s_lec10_slide_28.png

Example CTC results. No word lexicon or LM (WSJ corpus)

assets/New-Lecture-STT/cs224s_lec10_slide_29.png

CTC loss function

- Labels at each time index are **conditionally independent** given the audio input (like HMMs):

$$\Pr(a \mid x) = \prod_{t=1}^{T} \Pr(a_t \mid x)$$

- **CTC loss:** Negative log probability of the target transcription y^*, summed over all possible time-level labelings a that collapse to y^*:

$$\mathrm{CTC}(x) = -\log \Pr(y^*|x), \qquad \Pr(y|x) = \sum_{a \in B^{-1}(y)} \Pr(a|x)$$

- For sequence length $T = 3$, transcript HI:
  - All possible pre-collapsed/time-level labelings: `HI`, `H_I`, `_HI`, `HHI`, `H_II`
- The final loss maximizes the probability of the transcript y^*.
- **Insights:** Conditional independence allows us to compute outputs for each t in parallel.
  - Blank + collapsing enables efficient, exact summation over all alignments.

CTC inference: Most likely alignment

- **Output model assumes conditional independence.**
- Each t most likely output depends only on inputs (not auto-regressive).
- Most likely pre-collapsed output is simply the sequence of argmax outputs for each t.
- Run CTC collapsing to obtain transcript.

assets/New-Lecture-STT/cs224s_lec1C

Improving on CTC: RNN-Transducer (RNN-T) output model

- Add network that reasons over sequence of non-blank characters/tokens and input sequence
- Decoding outputs each transcript token only once. Optional blanks between. Variation on CTC
- Prediction network often pre-trained with CTC. Encoder provides transformed features for each t

assets/New-Lecture-STT/cs224s_lec10_slide_47.png

Self-Supervised Learning: Wav2Vec 2.0

Wav2Vec 2.0: Overview

wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations

Alexei Baevski **Henry Zhou** **Abdelrahman Mohamed** **Michael Auli**

{abaevski,henryzhou7,abdo,michaelauli}@fb.com

Facebook AI

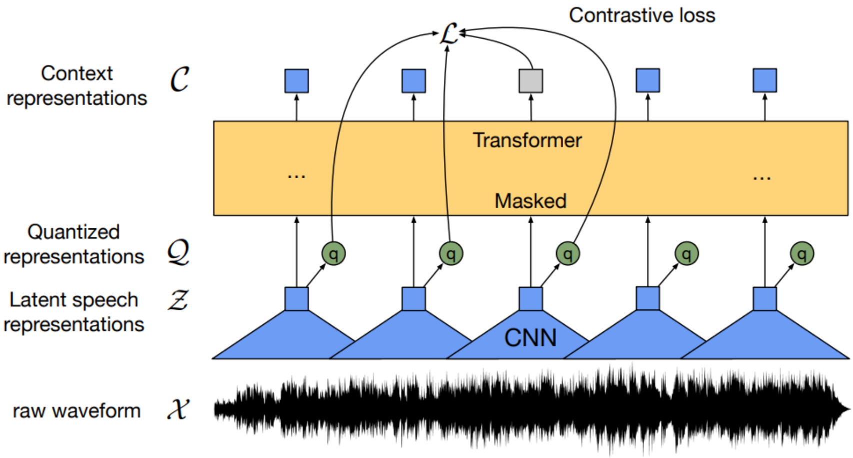
Abstract

We show for the first time that learning powerful representations from speech audio alone followed by fine-tuning on transcribed speech can outperform the best semi-supervised methods while being conceptually simpler. wav2vec 2.0 masks the speech input in the latent space and solves a contrastive task defined over a quantization of the latent representations which are jointly learned. Experiments using all labeled data of Librispeech achieve 1.8/3.3 WER on the clean/other test sets. When lowering the amount of labeled data to one hour, wav2vec 2.0 outperforms the previous state of the art on the 100 hour subset while using 100 times less labeled data. Using just ten minutes of labeled data and pre-training on 53k hours of unlabeled data still achieves 4.8/8.2 WER. This demonstrates the feasibility of speech recognition with limited amounts of labeled data.[1]

Wav2Vec 2.0: Overview (cont.)

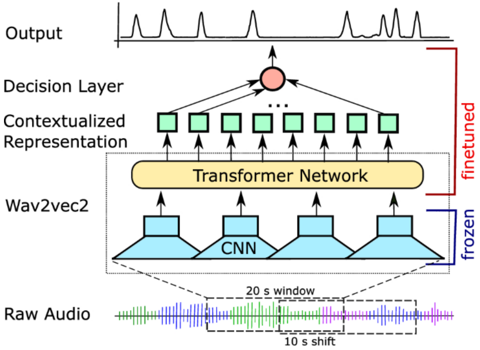
- **Proposed by Baevski et al. (2020)**
- **Key Idea:** Self-supervised learning for speech representation.
- **Architecture:**
  - CNN encoder for raw waveform $\rightarrow$ latent representations z_t
  - Transformer for context representations c_t
  - Quantization module for discrete representations q_t

Wav2Vec 2.0: Overview (cont.)



Wav2Vec 2.0 architecture overview.

- **Step 1:** Raw audio waveform is input to a multi-layer CNN encoder.
- **Output:** Latent speech representations z_t.



$$\text{Waveform} \xrightarrow{\text{CNN}} z_t$$

Wav2Vec 2.0: Contextualization and Masking

- **Step 2:** Random spans of z_t are masked.
- **Step 3:** Transformer network produces contextualized representations c_t.

$$z_t \xrightarrow{\text{mask spans}} \xrightarrow{\text{Transformer}} c_t$$

- **Step 4:** Quantize z_t to discrete representations q_t.
- **Quantization:** Product of G codebooks, each of size V.

$$z_t \xrightarrow{\text{Quantization}} q_t$$

- **Contrastive Loss:** Distinguish true quantized vector q_t^+ from negatives q^-.

$$L_t = -\log \frac{\exp\big(\mathrm{sim}(c_t, q_t^+)\big)}{\sum_{q^-} \exp(\mathrm{sim}(c_t, q^-))}$$

- **Diversity Loss:** Encourages usage of all codebook entries.

Wav2Vec 2.0: Performance

- **State-of-the-art results:**
  - ∼1.8% WER with 960h labeled data
  - ∼4.8% WER with only 10 minutes labeled data
- **Impact:** Enables efficient use of unlabeled speech data.

TIMIT phoneme recognition accuracy in terms of phoneme error rate (PER).

	dev PER	test PER
CNN + TD-filterbanks [59]	15.6	18.0
PASE+ [47]	-	17.2
Li-GRU + fMLLR [46]	–	14.9
wav2vec [49]	12.9	14.7
vq-wav2vec [5]	9.6	11.6
This work (no LM)		
LARGE (LS-960)	7.4	8.3

- After CTC/RNN-T hybrids, **Conformer** encoders and **Whisper**-scale weak supervision define current practice.

Conformer: Convolution-augmented Transformer for Speech Recognition

- Sequence-to-sequence transformer with multi-headed self attention.
- Combines attention (global context) with convolution (local invariance).
- Uses RNN-T loss architecture.

assets/New-Lecture-STT/cs224s_lec11_slide_04.png

- Mask time/frequency bands in log-mel during training for robustness.

assets/New-Lecture-STT/cs224s_lec11_slide_05.png

Convolution subsampling: Log mel spectrogram is an image

assets/New-Lecture-STT/cs224s_lec11_slide_06.png

The convolution block has a greatest effect on performance

- From ablation experiment of the paper, **removing unique features and moving towards a vanilla Transformer block** was tested.
- All ablation study results are evaluated without external LM.

The convolution block has the greatest effect on performance.

assets/New-Lecture-STT/cs224s_lec1

Ablation results: Largest WER drop from convolutions.

State-of-the-art ASR: Whisper

assets/New-Lecture-STT/cs224s_lec11_slide_18.png

Whisper: How does it perform a variety of tasks accurately?

assets/New-Lecture-STT/cs224s_lec11_slide_20.png

Whisper: encoder–decoder architecture

assets/New-Lecture-STT/cs224s_lec11_slide_21.png

Whisper encoder: transformer encoder

- Encoder input: log-mel spectrogram and two convolutional layers
- Positional embeddings
- Standard transformer blocks depending on model size

CS 224S / LINGUIST 285

Lecture 11: State-of-the-art deep learning approaches for speech recognition.

assets/New-Lecture-STT/cs224s_lec11_slide

Whisper decoder: autoregressive transformer decoder

- **GPT-2 style** model: no more CTC loss.
- Decodes using **auto-regressive prediction**—each token depends on previous (text prediction).
- **Cross-attention:** To encoded audio representation from encoder.
- **Custom vocabulary and tags:**
  - Language tags: <en>, <fr>, etc.
  - Task tags: <transcribe>, <translate>
  - **Timestamps**
  - Speech tags: <nospeech>

assets/New-Lecture-STT/cs224s_lec11_s

Whisper handles long-form transcription

Whisper handles long-form transcription

- Whisper only takes 30 seconds of speech at a time
- **Timestamp tokens** are used to determine when to shift the window
- Text from the previous window is provided with transcribing current window
- Voice activity detection using `<nospeech>` tag

assets/New-Lecture-STT/cs224s_lec11_slide_

Whisper's automatic speech translation performance

assets/New-Lecture-STT/cs224s_lec11_slide_34.png

Speech translation performance on FLEURS dataset (higher is better)

- Translation performance is better on average after the 100 hours of training data mark.
- Amount of training data isn't as correlated with translation performance as it is with transcription performance.
- Although possible, speech translation is likely to only be reliable for high resource languages.

What to consider when choosing an ASR solution

- **How many hours of data do you have?**
  - If you have hundreds of hours of data and need speech recognition for a specific domain, you could get away with training your own model or fine tuning an existing model.
  - If you have tens of hours of data, you can finetune an existing model.
- **Which language are you looking to transcribe or translate?**
  - If the model is well represented in Whisper, it is a strong baseline, especially for translation into English.
  - If you'd like to translate into another language, you will need your own model.

What to consider when choosing an ASR solution (cont.)

- **What is your compute budget?**
  - If you don't have any compute budget, using the best model you can find may be the best way forward.
  - If you have a small compute budget, finetuning a small model variant could outperform a larger model out-of-the-box.
  - If you have a large compute budget (and large enough dataset) you can consider finetuning a larger model variant.

Real-world factors and fairness

- Noise, codecs, mic quality, overlap, and domain mismatch dominate field WER.
- Demographic bias in accents and dialects—audit before justice, hiring, or grading use cases.
- Voice is biometric: consent, retention, on-device options, and transparent API error rates.

Benchmarks and open tooling

- **Datasets:** LibriSpeech, GigaSpeech, Common Voice, AMI, CHiME, Fleurs—match to deployment domain.
- **Toolkits:** ESPnet, NeMo/Riva, Fairseq, SpeechBrain, Hugging Face `transformers`.

References

1. Stanford CS224S (2025): lec9 (classical ASR), lec10 (neural acoustic modeling), lec11 (Conformer, SpecAugment, Whisper).
2. Edinburgh ASR course (2025): lectures-2025.
3. CMU 11-751/11-752 speech recognition: WAVLab.
4. JHU CLSP intro HLT slides; MIT 6.345/6.541J lecture notes (signal production, perception).
5. Graves et al., CTC, ICML 2006: icml_2006.pdf.
6. Gulati et al., Conformer, Interspeech 2020: arXiv:2005.08100.
7. Radford et al., Whisper, 2022: arXiv:2212.04356.
8. Baevski et al., wav2vec 2.0, 2020: arXiv:2006.11477.
9. Chan et al., Listen Attend and Spell, 2015: arXiv:1508.01211.
10. Graves, Sequence Transduction (RNN-T), 2012: arXiv:1211.3711.

11. Hinton et al., DNN acoustic modeling, 2012: DNN-2012-proof.pdf.
12. Mohri et al., WFSTs in speech recognition; Distill CTC; Hugging Face ASR evaluation docs [47].